



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7 **по курсу «Компьютерные сети»**

**«Информационная система мониторинга блоков в блокчейн
Ethereum»**

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Реализовать приложение мониторинга состояния заданных блоков блокчейн Ethereum. Данные результатов мониторинга должны записываться в Firebase. Аккаунт в infura.io необходимо зарегистрировать свой, также создать собственную Realtime Database.

Необходимо показать значения транзакций в консоли, сравнить их с данными представляемыми сайтом <https://etherscan.io/> — должны совпадать. Транзакции должны записывать в Firebase и обновляться, после обновления данных в блоках.

2 Результаты

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.go

```
1 package main
2
3 import (
4     "bytes"
5     "context"
6     "encoding/json"
7     "fmt"
8     "log"
9     "math/big"
10    "net/http"
11    "time"
12
13    "github.com/ethereum/go-ethereum/ethclient"
14 )
15
16 // Block data structure
17 type BlockData struct {
18     Number          uint64      'json:"number"'
19     Time            uint64      'json:"time"'
20     Difficulty      uint64      'json:"difficulty"'
21     Hash            string      'json:"hash"'
22     Transactions_count int        'json:"transactions_count"'
23     Transactions    []TransactionData 'json:"transactions"'
24 }
25
```

```

26 // Transaction data structure
27 type TransactionData struct {
28     Hash      string 'json:"hash"'
29     To        string 'json:"to"'
30     Value     string 'json:"value"'
31     Gas       uint64 'json:"gas"'
32     GasPrice  string 'json:"gasPrice"'
33 }
34
35 func main() {
36     // Connect to Ethereum via Infura
37     client, err := ethclient.Dial("https://mainnet.infura.io/v3/
        d1eccb6a8c334a47b8e6635379bb4fe8")
38     if err != nil {
39         log.Fatalf("Error connecting to Ethereum: %v", err)
40     }
41
42     // Prepare Firebase endpoint
43     firebaseURL := "https://lab7-f46db-default-rtdb.europe-west1.
        firebaseDATABASE.app/blocks/latest.json" // Your database URL
44     apiKey := "AIzaSyDnboKcMq5mhnBQebKrxhFuyI1lh0aM9M" // Firebase API
        key
45
46     var latestBlock uint64
47
48     for {
49         // Get the latest block
50         header, err := client.HeaderByNumber(context.Background(), nil)
51         if err != nil {
52             log.Fatalf("Error retrieving the latest block: %v", err)
53             time.Sleep(5 * time.Second)
54             continue
55         }
56
57         currentBlock := header.Number.Uint64()
58         if currentBlock > latestBlock {
59             // Update the latest known block
60             latestBlock = currentBlock
61
62             // Retrieve information about the new block
63             blockNumber := big.NewInt(header.Number.Int64())
64             block, err := client.BlockByNumber(context.Background(),
                blockNumber)
65             if err != nil {
66                 log.Fatalf("Error retrieving block data: %v", err)
67             }

```

```

68
69 // Collect transaction data from the block
70 var transactions []TransactionData
71 for _, tx := range block.Transactions() {
72     toAddress := ""
73     if tx.To() != nil {
74         toAddress = tx.To().Hex()
75     }
76
77     transactions = append(transactions, TransactionData{
78         Hash:      tx.Hash().Hex(),
79         To:         toAddress,
80         Value:      tx.Value().String(),
81         Gas:        tx.Gas(),
82         GasPrice: tx.GasPrice().String(),
83     })
84 }
85
86 // Format block data
87 blockData := BlockData{
88     Number:      block.Number().Uint64(),
89     Time:        block.Time(),
90     Difficulty:   block.Difficulty().Uint64(),
91     Hash:        block.Hash().Hex(),
92     Transactions_count: len(block.Transactions()),
93     Transactions: transactions,
94 }
95
96 fmt.Printf("Block data retrieved: %v\n", blockData.Number)
97
98 data, err := json.Marshal(blockData)
99 if err != nil {
100     log.Fatalf("Error converting block data to JSON: %v", err)
101 }
102
103 req, err := http.NewRequestWithContext(context.Background(), "POST",
104     firebaseURL+"?auth="+apiKey, bytes.NewBuffer(data))
105 if err != nil {
106     log.Fatalf("Error creating HTTP request: %v", err)
107 }
108
109 req.Header.Set("Content-Type", "application/json")
110 clientHTTP := &http.Client{}
111 resp, err := clientHTTP.Do(req)
112 if err != nil {
113     log.Fatalf("Error sending data to Firebase: %v", err)

```

```

113         time.Sleep(5 * time.Second)
114         continue
115     }
116     defer resp.Body.Close()
117
118     if resp.StatusCode == http.StatusOK {
119         fmt.Println("Data successfully written to Firebase!")
120     } else {
121         log.Fatalf("Error writing data to Firebase: %v", resp.Status)
122     }
123 }
124 time.Sleep(5 * time.Second)
125 }
126 }

```

Результат запуска представлен на рисунках 1– 2.

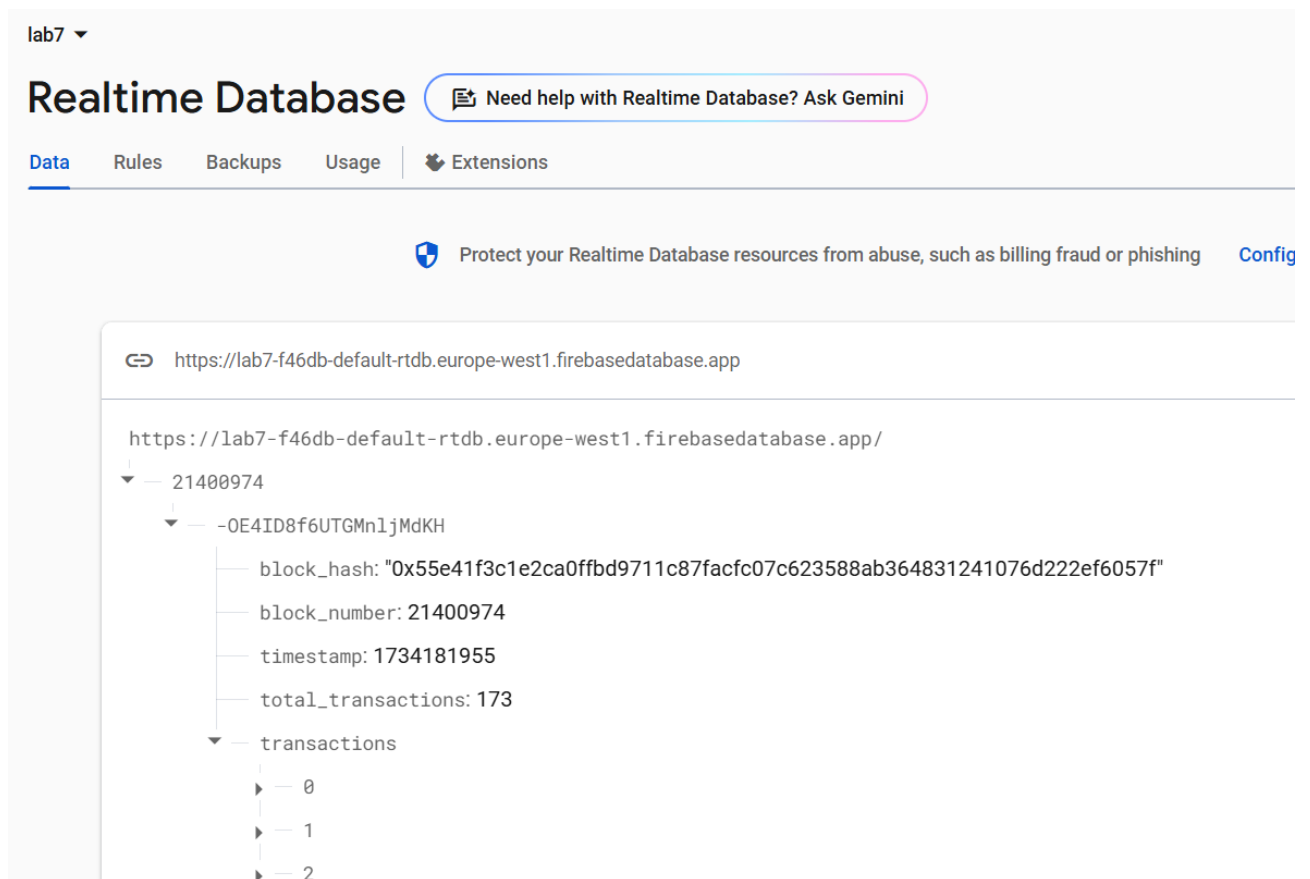


Рис. 1 — Результат со стороны Firebase

```
-----  
Number: 21401073  
Hash: 0xe66cf64918572dc4d551e39239fd32b6f66bbdecdf6bb922f2a0c8f35ba10f7  
Timestamp: 1734183143  
Total transactions: 113  
Блок 21401073 успешно сохранен  
-----  
Number: 21401074  
Hash: 0x374e3c4b1fb60e47563f0bf12a1c676a7a4ac52583215bb96c60974ae0da1ab2  
Timestamp: 1734183155  
Total transactions: 256  
Блок 21401074 успешно сохранен
```

Рис. 2 — Результат со стороны консоли